Nonlinear Dyn
https://doi.org/10.1007/s11071-021-06491-3

ORIGINAL PAPER

Designing an authenticated Hash function with a 2D chaotic map

Luis Gerardo De la Fraga · **Cuauhtemoc Mancillas-López** · **Esteban Tlelo-Cuautle**

Received: 8 September 2020 / Accepted: 23 April 2021
© The Author(s), under exclusive licence to Springer Nature B.V. 2021

Abstract This paper shows the use of four sets of coefficient values of the 2D chaotic map to generate pseudo-random number generators. We demonstrate that the generated sequences are random by applying NIST suite 800-22-a and TestU01 tests. The generated random sequences are used to implement a stream cipher and are applied to encrypt images. To detect if the images have been modified, we propose to use the random sequences as keys for a hash function based on the pseudo-dot product. This hash can be used as a message authentication code in the images to detect if the stored information has been compromised. The proposed schemes can be used to encrypt and authenticate any digital data not only images. The random sequences generator is probed also in a high-performance microcontroller STM32F746ZG obtaining a throughput of 173.35 Kbit/s.

Keywords 2D chaotic map · Data integrity · Keyed hash · Stream cipher · Chaos-based cryptography

L. G. de la Fraga (✉)
Computer Science Department, Cinvestav, Av. IPN 2508, 07360 Mexico City, Mexico
email: fraga@cs.cinvestav.mx

C. Mancillas López
Computer Science Department, Cinvestav, Av. IPN 2508, 07360 Mexico City, Mexico
email: cuauhtemoc@cs.cinvestav.mx

E. Tlelo-Cuautle
Department of Electronics, INAOE, Luis Enrique Erro No.1, 72840 Tonantzintla, Puebla, Mexico
email: etlelo@inaoep.mx

1 Introduction

For some years, the term "image encryption" [10, 17,22,26,33] is used to indicate the application of chaotic entities—mostly maps—to secure images by encrypting them. The justification to image encryption is that images have certain characteristics as data redundancy, strong correlation among adjacent pixels, being less sensitive as compared to the text data, etc. [1,9,11,13,17,25,26,31]. In this work, we demonstrate that the security to encrypt and generate a message authentication code (MAC), or an authentication tag, for an image does not depend on any image characteristic, or special distribution of the image data. The security to encrypt an image depends on the random sequences generated with a given chaotic entity.

Also, in articles related to image encryption authors do not present a security solution if the stored encrypted data of the images, or the transmitted data image, are modified [1,9,11,13,25,31]. We claim that the only way to detect if an image (or any information) has been modified is by creating an image integrity function. This function calculates an authentication tag for the image, encrypts it, and stores it next to the input image. Each time that it is necessary to check if the stored information is not compromised, the authentication tag must be calculated again from the data and compared with the stored one. As will be shown, the security to calculate an authenticated tag depends on

Published online: 12 May 2021

the choice of a function to generate such authenticated tag.

Chaotic maps have been used to create pseudo-random number generators (PRNGs) [7,17]. The chaotic maps are a source of entropy to the PRNGs. The logistic map was used in [26]. The Bernoulli shift map, tent map, and zigzag map were used in [7]. A 4D chaotic hyperjerk system was used in [31]. Hénon, Tinkerbell, Chen, Rössler, and logistic maps were used in [11]. The Ikeda system was used in [17]. The security of random sequences generated with the 2D Cat map is studied in [19]. Also several of these proposals have been implemented in hardware, specially in FPGAs [7] to increase the PRNG's throughput.

Related research areas of this work could be on universal hash functions, data integrity and authentication [2,4,16].

Three are the main contributions of our work:

- The use of the 2D chaotic map, which utilizes only integer sums and multiplications, to generate a sequence of random binary numbers. To our best knowledge, this map has not been used previously to generate random numbers.
- The second contribution: we propose to encrypt image data with the xor operation among image bits and the generated random sequence. Indeed, this procedure can be used to encrypt any digital data, not only images.
- Our third contribution is to use a chaotic map as the basis to generate a keyed hash function, in which the key is a long sequence of random numbers, for image integrity in its transmission and encryption. The keyed hash can detect any change applied to the original or the encrypted image. Also, to the best of our knowledge, it is the first time that a chaotic map is used as the core to implement a keyed hash function.

Other authors have used to encrypt an image a two steps procedure of confusion and diffusion of the image pixels [15,21,23,32]. We propose in our article that the simplest way of xoring the image bits with the generated binary random sequence is required to encrypt the image. The requirement is that the binary sequence must be random. If data (not only an image) are xoring with a random sequence, the resulted sequence is also random. We will show this fact with two experiments.

Furthermore, our proposal scheme can solve common insecurity problems described in [20] and [18]. In specific, we are not using floating point arithmetic—which avoid quantization problems—the image pixels are not permutated—which could be used by plain text security attacks—and 64 bits are used in the calculations—which leads both to avoid low periods and degradation in the used map, and to use wider keys.

This document is organized as follows: In Sect. 2, the 2D chaotic map, and how random numbers are generated, are described. In Sect. 3, the procedure to encrypt images (or any other digital information) is presented. The integrity of the images (or any other digital message) is presented in Sect. 4. In Sect. 5 are described the two experiments to show that the encrypted image cannot be distinguished from random noise. In Sect. 6, a brief discussion of this work is presented. Finally, in Sect. 7, some conclusions are presented.

2 2D chaotic map

This map was proposed by Sprott [30]. The equations for the bidimensional chaotic map are defined by

$$\begin{aligned} x_{n+1} &= a_1 + a_2 x_n + a_3 x_n^2 + a_4 x_n y_n + a_5 y_n + a_6 y_n^2, \\ y_{n+1} &= a_7 + a_8 x_n + a_9 x_n^2 + a_{10} x_n y_n + a_{11} y_n + a_{12} y_n^2. \end{aligned} \tag{1}$$

In Eq. (1), it is possible to see that this simple 2D chaotic map uses only sums and multiplications. Other authors have used more complex 2D chaotic maps, for example, in [21] authors use the Henon map and the 2D Chebyshev map; and the 2D Chebyshev map uses the cosine and arccosine function. In [15], authors use two logistic maps and two-dimensional baker map; this last one map uses one division operation. In [23], authors use a two-dimensional Sine ICMIC modulation map, which uses cosine function. The map used in [32] required a sine function.

In [30], the values of the coefficients for (1) were searched randomly within the range $[-1.2, 1.2]$, with an increment of 0.1, to search for strange attractors. From [30], we choose the following sets of coefficient values $\{a_1, a_2, \ldots, a_{12}\}$:

$$\begin{aligned} M_1 &= \{-0.6, -0.1, 1.1, 0.2, -0.8, 0.6, \\ &\quad -0.7, 0.7, 0.7, 0.3, 0.6, 0.9\}, \\ M_2 &= \{-1.0, 0.9, 0.4, -0.2, -0.6, -0.5, \\ &\quad 0.4, 0.7, 0.3, -0.5, 0.7, -0.8\}, \end{aligned}$$

Table 1 Lyapunov exponent and fractal dimension values (taken from [30]) of the used 2D map

Map	Lyapunov exponent	Fractal dimension
1	0.12	1.77
2	0.14	1.79
3	0.15	1.69
4	0.13	1.50

Table 2 Number of bits used in the implementation of each map with integer arithmetic

Map	Bits integer part	Bits fractional part
1	3	60
2	4	59
3	4	59
4	3	60

1 bit is used as bit sign

$$M_3 = \{0.8, 1.0, -1.2, -1.0, 1.1, -0.9, 0.4, -0.4, -0.6, -0.2, -0.5, -0.7\},$$
$$M_4 = \{-0.6, -0.4, -0.4, -0.8, 0.7, 0.3, -0.4, 0.4, 0.5, 0.5, 0.8, -0.1\}.$$

These sets of values correspond to the maps in [30] with greater Lyapunov exponent and fractal dimension values (see Table 1).

The phase portraits (in black) and their domains of attraction (in gray) of the four maps are shown in Fig. 1.

The 2D map could suffer of degradation if it is implemented in integer arithmetic with fewer than 18 bits [6]. We implement the four maps in integer arithmetic using 64-bit numbers, which is far beyond of suffering any degradation. The number of bits in the integer part and in the fractional part for each set of coefficient values of the 2D chaotic map is shown in Table 2.

A sequence of pseudo-random bits can be generated in the following form. Given the initial values x_0, y_0, the following numbers x_{n+1} and y_{n+1} are calculated using (1), and at each $n+1$ iteration 16 random bits of the sequence s are generated as

$$s_{n+1} = \{x_{n+1} \bmod 256,\ \ y_{n+1} \bmod 256\}. \tag{2}$$

For simplicity, a region of range equal to 1 has choice within the domain of attraction to initialize x_0 and y_0 for each map. These region values are shown in Table 3

Table 3 Used ranges for the initial values for all four maps

Map	Ranges of values for x_0 and y_0
M_1	$x_0 \in [-0.5, 0.5]$, $y_0 \in [-0.5, 0.5]$
M_2	$x_0 \in [-1.0, 0.0]$, $y_0 \in [0.0, 1.0]$
M_3	$x_0 \in [0.0, 1.0]$, $y_0 \in [-0.6, 0.4]$
M_4	$x_0 \in [-1.5, -0.5]$, $y_0 \in [-0.5, 0.5]$

and were chosen seeing the graphs in Fig. 1. Coefficient values were initialized using their corresponding real double precision numbers; thus, they have about 55 precision bits.

If the limits for x_0 and y_0 are within the range [0, 1], and we are using 59 bits for the fractional part, then this same number of bits could be used to form the key; thus, the key length could be 118 bits. But the 3 last significant bits do not change the generated sequence; thus, only 55 bits can be used for x_0 or y_0, and the key is the concatenation of both values; therefore, the key is 110 bits length.

2.1 NIST and TestU01 tests

The National Institute of Standards and Technology (NIST) of the USA had created a free software for testing pseudo- and random number generators [(P)RNGs]. Specifically, NIST said that the test is for (P)RNGs for cryptographic applications, where randomness is a crucial characteristic [27].

Three tests of TestU01 suite were used: Rabbit, Alphabit, and BlockAlphabit. These tests were designed for bits stored in a file (or a physical device). Rabbit and Alphabit apply 38 and 17 different statistical tests, respectively. BlockAlphabit applies the Alphabit battery of tests repeatedly to a binary file after reordering the bits by blocks of different sizes (with sizes of 2, 4, 8, 16, 32 bits) [8].

We applied the 15 NIST, and 3 TestU01 tests to 100 sequences of 10^6 bits; these sequences were generated with the four 2D maps (see Fig. 1). The same initial condition of $x_0 = 0.1$ and $y_0 = 0.2$ was used to all the maps. The used numbers are shown in Table 2. The sequences passed all NIST and TestU01 tests (see Table 4).

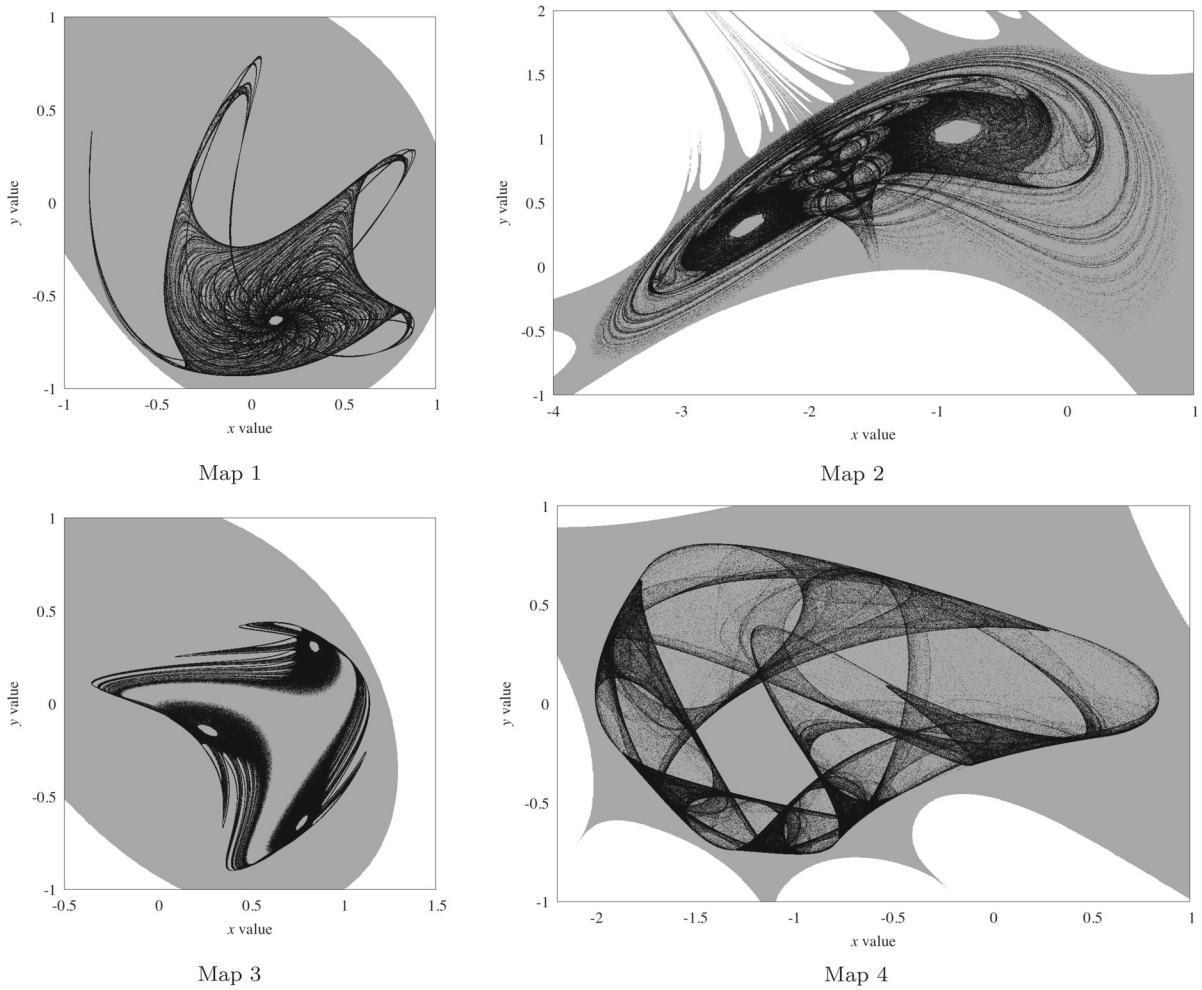


Fig. 1 The four chaotic attractors used in this work and their domains of attraction in floating-point arithmetic

2.2 PRNG implementation in a microcontroller

The PRNG was implemented in a high-performance microcontroller. The development board Nucleo F746ZG was used. It has STM32F746ZGT6 MCU with up to 216 MHz Cortex-M7F core, 1024 KB flash, and 336 KB SRAM. This MCU is 32 bits; thus, the multiplication of two 64-bit numbers was coded using 32-bit multiplications.

The measured throughput was of 12.84 Kbits/s at the MCU working frequency of 16 MHz, or it could be 173.35 Kbits/s working at the maximum clock frequency of 216 MHz.

All codes for the 2D maps are publicly available at http://cs.cinvestav.mx/~fraga/2DMaps.tar.gz.

3 Image encryption

Given any digital message, this is, an information composed by bits, it can be encrypted using the described chaotic maps. Any of the four implementations with integer arithmetic of the 2D chaotic map generate at each iteration (at time $n+1$) 16 bits. Then, it is possible to generate sequences of random bits at multiples of 16 bits; thus, our word length is of 16 bits. We can define a function called StreamCipher [24], which receive as input a message of size n bits; this function internally generates $\lceil n/16 \rceil$ random words and apply xor operation between the message bits and the generated random words. The pseudocode to encrypt is given in Algorithm 1. The initialization vector (IV) is used

Table 4 Results of applying NIST and TestU01 tests to our PRNG implemented in fixed point arithmetic

	Test name	Map 1		Map 1		Map 3		Map 4	
		p value	Prop.	p value	Prop.	p value	Prop.	p value	Prop.
1	Frequency	0.616305	0.99	0.030806	0.98	0.637119	1.00	0.474986	0.99
2	Block frequency	0.013569	0.99	0.897763	0.98	0.102526	0.99	0.419021	0.98
3	Cumulative sums	0.843604	1.00	0.368835	0.99	0.534272	1.00	0.516925	0.99
4	Runs	0.616305	1.00	0.115387	0.99	0.924076	0.98	0.494392	1.00
5	Longest run	0.514124	0.98	0.383827	1.00	0.514124	1.00	0.834308	1.00
6	Rank	0.419021	0.99	0.883171	1.00	0.275709	0.97	0.657933	1.00
7	FFT	0.971699	1.00	0.851383	0.96	0.883171	0.97	0.171867	0.99
8	Nonoverlapping template	0.539914	0.99	0.485793	0.99	0.468335	0.99	0.459617	0.99
9	Overlapping Template	0.051942	0.99	0.816537	0.99	0.719747	1.00	0.514124	0.99
10	Universal	0.304126	0.99	0.145326	0.96	0.851383	0.97	0.224821	1.00
11	Approximate entropy	0.000253	0.99	0.025193	0.99	0.040108	0.97	0.275709	0.99
12	Random excursions	0.354881	0.99	0.379257	0.99	0.412028	0.99	0.694025	0.98
13	Random excursions variant	0.416729	0.99	0.444640	1.00	0.563374	0.99	0.522156	0.99
14	Serial	0.237053	0.99	0.556935	0.99	0.336241	0.99	0.633499	0.99
15	Linear complexity	0.759756	1.00	0.224821	1.00	0.020548	0.96	0.911413	1.00
16	Rabbit	All 40 tests passed							
17	Alphabit	All 17 tests passed							
18	BlockAlphabit	All 6 repetitions of alphabit tests passed							

to avoid that encrypts the same message producing the same resulted sequence of bits.

Input: M_1 message (image) to encrypt, K the key, IV initialization vector.
Output: The encrypted M_1 message.
1: **function** ENCRYPT(M_1, K, IV)
2: R = StreamCipher(K, IV) ▷ Stream cipher using key K on IV
3: $K' = K \oplus R$ ▷ The key modified by R
4: M_2 = StreamCipher(K', M_1) ▷ M_2, the encrypted message
5: **return** M_2
6: **end function**

Algorithm 1: Pseudocode for the encryption function

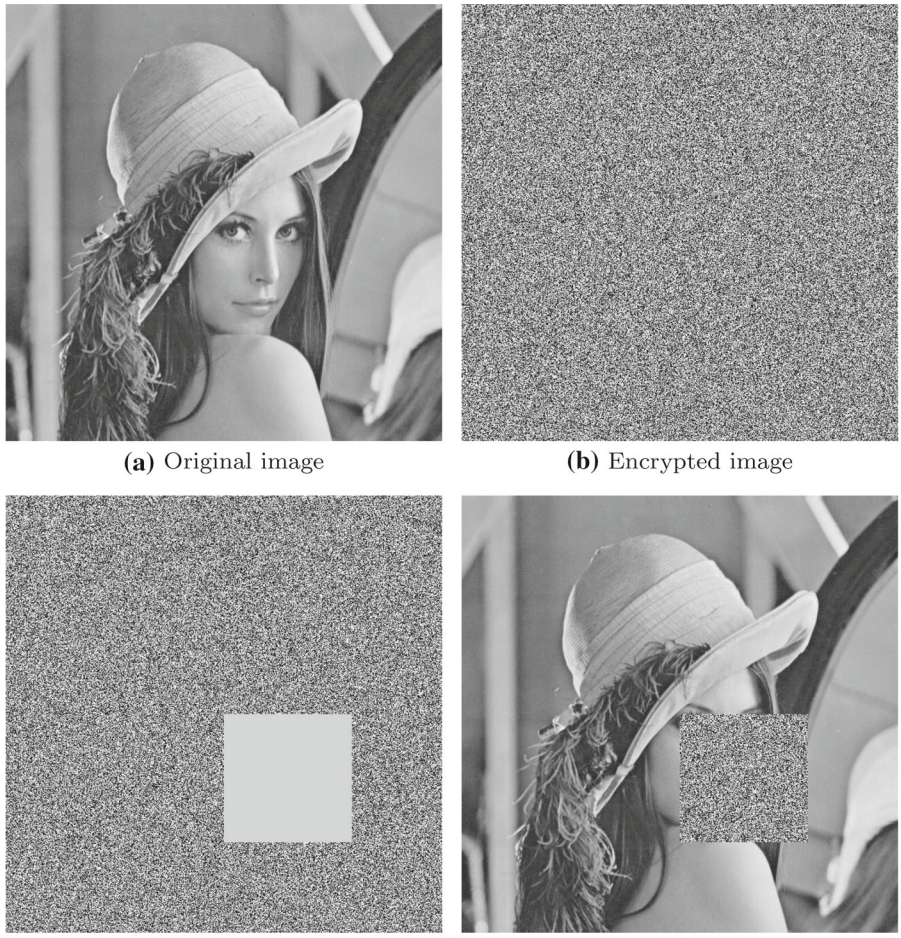
In line 1 in Algorithm 1, the IV is encrypted with the StreamCipher function using the key K; then, in line 2, a new key K' is generated by xoring the previous key an the encrypted IV. The input message is encrypted using this new key again using the StreamCipher function. The generated M_2 message (in line 3) is returned. The vector initializator (VI) or nonce, will generate a ciphered message different when a same key is used over two equal messages (or images).

The function to decrypt is exactly the same that to encrypt. Then, Algorithm 1 is also the pseudocode for the decrypt function.

An example of the encryption process using the image of Lena (in Fig. 2a, 512×512 pixels size) is shown in Fig. 2b. Applying Algorithm 1, with the same key, to image in Fig. 2b, the original image in Fig. 2a is obtained. If the encrypted image is modified as it is shown in Fig. 2c, then the complete original image cannot be recovered as it is shown in Fig. 2d. In this Fig. 2d we can see that only a partial information of the original image is recovered, and this is not a desirable. Here, we are proposing to authenticate the image building a keyed hash function, which also uses the random sequences generated with the 2D map.

4 Image integrity

The Algorithm 2 performs message (or image) encryption and data integrity. A *message authentication code* (MAC) is generated by applying a hash function over the input message (line 2, in Algorithm 2), using a key K and a random sequence provided by a 2D map.



(a) Original image **(b)** Encrypted image

(c) Modified encrypt image **(d)** Decrypted image of (c)

Fig. 2 One example of the encryption of image (**a**), in (**b**). In **c** the encrypted image is modified, **d** decrypted, **c** image

This hash function was designed by us, and a detailed description will be given in the next section. A common characteristic of a hash function is that it must give a short fixed length output; with very similar messages, their hash must be very different and, more important, should be impossible to generate a message that produces a given hash.

In Algorithm 2, the hash function uses the same key that the input key, but another different key could be used. The MAC is generated with the output hash xored with the encrypted IV (line 3 in Algorithm 2). This MAC is stored at the end of the messages (in line 4). A new key is generated by xoring the input key and the encrypted IV (line 5). With this new key, the entire message and MAC are encrypted using the StreamCipher function (line 6) which uses the bits generated by the used 2D map.

Input: M_1, the input message, K the key, a nonce IV
Output: M_3, the encrypted message of M_1.
1: R = StreamCipher(K, IV) ▷ IV vector is encrypted with K key
2: H = Hash(K, M_1) ▷ Key K, M_1 is the input message
3: MAC = $R \oplus H$ ▷ Message authentication code
4: $M_2 = M_1 \parallel$ MAC ▷ This MAC will be stored at the file end
5: $K' = K \oplus R$ ▷ Key K modified by IV
6: M_3 = StreamCipher(K', M_1) ▷ M_3 is the encrypted message

Algorithm 2: Pseudocode for the integrity function

If an attacker changes the stored (or transmitted) information, it can be verified when the message (or image) is decrypted. The on-the-fly generated MAC of the image must be equal to the stored MAC. If they are different, this means that the image has been modified by the attacker, the information is compro-

mised, and the image must not be used anymore. If the attacker modifies the stored MAC, the on-the-fly generated MAC will be different and this means also that the stored information is compromised. Remember that all the stored information is encrypted, and thus, the attacker could only modify the stored information bytes.

4.1 Hash calculation using the pseudo dot product

Our implementation of the hash function is based on the pseudo-dot product [3,5] (PDP). With the word length of 16 bits of our PRNG, and the key length of 110 bits, it fits 7 words for the length of the digest. The message M is composed of $M_1 \| M_2 \| \ldots \| M_m$ concatenated words, and another key Z is composed of $Z_1 \| Z_2 \| \ldots \| Z_{m+12}$ words. With these data and definitions, PDP is defined as

$$\mathrm{PDP}(M, Z) = h_1 \| h_2 \| \ldots \| h_7, \tag{3}$$

where the seven terms $h_1, h_2, \ldots, h_7$, each one of length 16 bits, are calculated as

$$\begin{aligned} h_1 =& (M_1 \oplus Z_1)(M_2 \oplus Z_2) \oplus (M_3 \oplus Z_3)(M_4 \oplus Z_4) \oplus \\ & \cdots \oplus (M_{m-1} \oplus Z_{m-1})(M_m \oplus Z_m), \\ h_2 =& (M_1 \oplus Z_3)(M_2 \oplus Z_4) \oplus (M_3 \oplus Z_5)(M_4 \oplus Z_6) \oplus \\ & \cdots \oplus (M_{m-1} \oplus Z_{m+1})(M_m \oplus Z_{m+2}), \\ & \ldots\ldots, \\ h_7 =& (M_1 \oplus Z_{13})(M_2 \oplus Z_{14}) \oplus \\ & \cdots \oplus (M_{m-1} \oplus Z_{m+11})(M_m \oplus Z_{m+12}). \end{aligned} \tag{4}$$

The key Z required by the PDP can be generated with a random sequence of the 2D chaotic map. All the multiplications in (4) produce a result of 32 bits and must be reduced to 16-bit words. A suitable number to reduce the multiplications is the modulus operation with a big prime number, and we decided to use the number $2^{16} + 1 = 0\text{x}10001$.

The implementation of PDP is in the pseudocode in Algorithm 3, where at the output the content of buffer h is the digest of size 112 bits of file M.

```
Input: Message M as an input file, K key
Output: Digest of M of size 7 · 16 = 112 bits.
1:                ▷ Used data: 2 buffers of 14 words (224 bits)
2: M[1 . . . 14]                                ▷ Buffer for the file
3: Z[1 . . . 14]                                     ▷ Key's buffer
4: h[1 . . . 7] = 0                  ▷ Buffer to store the digest
5: Z = 2Dmap(K, 14)                ▷ 14 words of random bits
6: while read 14 words from file M do
7:    for i = 1 : 7 do
8:       M1 = M[2 ∗ i − 1]                      ▷ 1 3 5 7 9 11 13
9:       M2 = M[2 ∗ i]                          ▷ 2 4 6 8 10 12 14
10:       here = 2 ∗ i
11:       next = here + 1
12:       for j = 1 : 7 do
13:          K1 = Z[here]
14:          K2 = Z[next]
15:          P1 = M1^K1
16:          P2 = M2^K2
17:          h[j] ^= (P1 ∗ P2) mod 0x10001U
18:          here += 2; here %= 14
19:          next += 2; next %= 14
20:       end for
21:       Z + 2 ∗ i = 2Dmap(K, 2)              ▷ 2 words of
   random bits
22:    end for
23:    for j = 1 : 7 do
24:       h[j] ^= 2Dmap(K, 2)
25:    end for
26: end while
```

Algorithm 3: Pseudocode for the digest calculation using the pseudo-dot product

4.2 Message authentication code

If we are interested only to know if the message (or image) has been modified, then the message is not encrypted, and an authentication tag is generated with the raw bits. This function is the same as in Algorithm 2 until the line 4: the MAC is encrypted using the input key K and IV and is stored at the file end. When one needs to verify the file, the same procedure in Algorithm 2 must be performed, but at the end the calculated on-the-fly MAC is compared with the stored and decrypted MAC; if both are equal, the file has not been modified. If both MACs are different, the stored information has been compromised.

In Fig. 3 are shown three images all of size 256×256 pixels; Fig. 3a is an entire gray tone image with gray value 128, and Fig. 3b is the same previous image with a black pixel (with value 0) in position [20,20]; the third image is the entire gray image with a black pixel in position [128,128]. These three images correspond to images 1, 2, and 4 in Table 5. The corresponding

(a) Gray tone image **(b)** Black pixel in [20,20] **(c)** Black pixel in [128,128]

Fig. 3 Three sample images to test the generation of authentication tags. The three images have size 256 × 256 pixels

authentication tags, the result of the PDP in Algorithm 3, shown in Table 5 are completely different. This is a desired characteristic of a hash function as PDP in Algorithm 3. Image 3 in Table 5 is like image 2, but the black pixel is in the next position [20,21]; visually, this image is equal to the image in Fig. 3b, but their authentication tags are also totally different. Similarly, image 5 in Table 5 is like image 4, but the black pixel is in the next position [128,129]; visually, this image is equal to image in Fig. 3c. Check their authentication tags; they are also totally different. Finally, image 6 in Table 5 is like image 4, but the black pixel is changed to the value of 1, this is, a single bit in the whole image is changed, and the resulted authentication tags are, again, totally different. A modification of any bit in the original image is easily detected by calculating its keyed hash. Also, the calculated digests are totally different, and it is a characteristic that makes impossible to recover any information of the image from its keyed hash.

The proposed keyed hash can be used also to check the integrity when one image is transmitted, without been encrypted. This is similar as the traditional hash functions are used as MD5 or SHA256 functions, with the difference that it needs a key. MD5 is used to check the integrity of Linux distribution when the huge DVD data are correctly downloaded from Internet, for example. The calculated keyed hash of the attacked image will be totally different of that the keyed hash of the original image, even when one single bit is changed. The keyed hash also can provide authentication because only the owners of the secret key can obtain the correct hash.

Table 5 Some examples of images and their MACs calculated with Algorithm 3

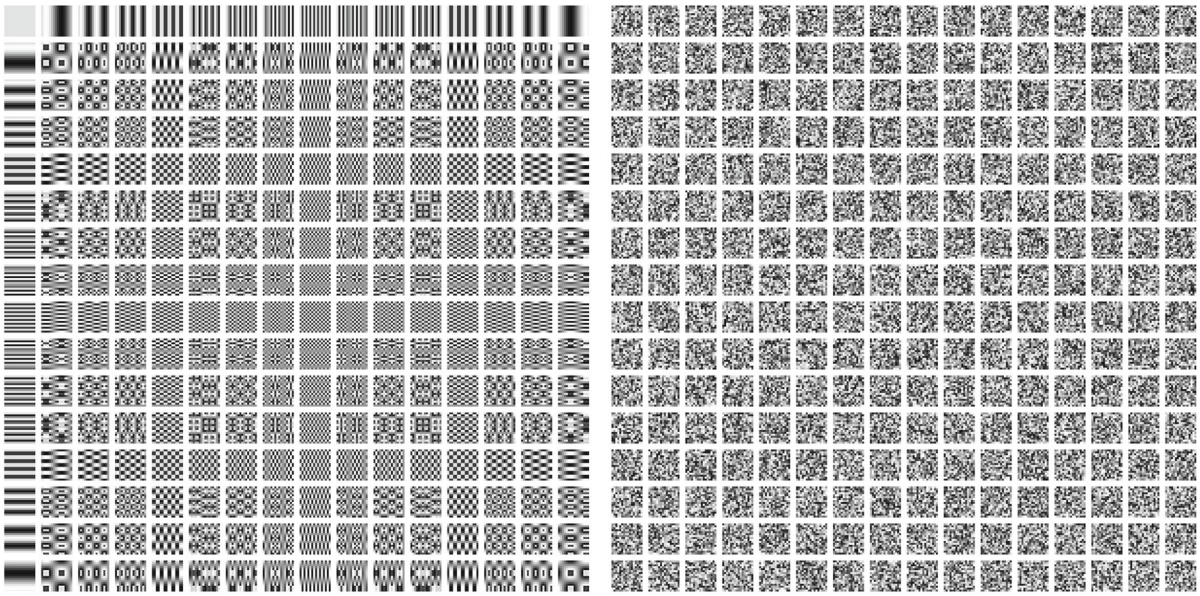
No.	Image	MAC
1	Gray image	0x5ea30a779f7eaf020cc6059a6aad
2	img[20, 20] = 0	0x69a516f5628d9659f2ced00c1554
3	img[20, 21] = 0	0x5c7488436035686a143cb340d5a6
4	img[128, 128] = 0	0x6024eb681f2fafbfd9fa1dda5feb
5	img[128, 129] = 0	0xdbd51656cdfe5003df9ac572549e
6	img[128, 128] = 1	0x6a9133a9bc2cf5a022539d8ac380

These authentication tags are not encrypted, and a same key was used to generate them

5 The encrypted image cannot be distinguished from noise

Any image can be decomposed into a weighted sum of orthogonal basic functions [12]. The basic functions, or basic images for a bidimensional function, for the discrete Hartley transform of size 16 × 16 pixels are shown in Fig. 4a. Other transformations are possible, as the Fourier, sine, cosine, Haar or wavelet transforms. The Hartley transform was chosen here because the basic images are real (or nonimaginary images). The Hartley basic images are generated [12] using the function s defined by

$$s(x, y, u, v) = \left[\frac{1}{\sqrt{n}}\text{cas}\left(\frac{2\pi ux}{n}\right)\right]\left[\frac{1}{\sqrt{n}}\text{cas}\left(\frac{2\pi vy}{n}\right)\right] \quad (5)$$



(a) The 256 basic images of the discrete Hartley transform for an image of 16 × 16 pixels.

(b) The encrypted images of the corresponding images in (a)

Fig. 4 All the 256 basic images of the discrete Hartley transform for an image of size 16 × 16 pixels in (**a**) and their corresponding encrypted images in (**b**). Here, we can observe that all the spatial information in any basic image in (**a**) is destroyed in the corresponding encrypted image in (**b**)

where cas function is defined as $\text{cas}(\theta) = \sin(\theta) + \cos(\theta)$.

The basic images are encrypted and shown in Fig. 4b. From all these images, one can observe that all the spatial information is lost by xoring the basic images with the generated chaotic random sequence.

To verify numerically that the encrypted images cannot be distinguished of random noise, another experiment was performed: the first basic images in the diagonal of Fig. 4a, s_{uv}, for $u \in \{0, 1, 2, 3, 4\}$ and $v = u$ were encrypted 100 times with different keys. The 100 encrypted images were averaged and their histograms were calculated. The random sequence is a random signal with an uniform distribution, with mean equal to 127.5 ((255-0)/2) and variance of $\sigma^2 = 255^2/12$. The theoretical signal in the averaged images is a Gaussian function with mean also equal to 127.5, and standard deviation $\sigma/\sqrt{100}$. The basic images, the respective average image, and the image of only noise are shown in Fig. 5; the histograms of the averaged images are shown in Fig. 6. In the histogram of Fig. 6, it can be seen that it is impossible to distinguish the encrypted signal from the noise, which indeed is the basis of the encryption process.

6 Discussion

The presented implementation of PRNG with the 2D chaotic map generates 16 random bits at each iteration. We propose to use a key of 110 bits that corresponds to a range within [0, 1] for the variation of starting x_0 and y_0 values using fixed point arithmetic. Of course, the 2D maps could be used in parallel to generate 2, 3, or 4 words of random bits at each iteration, and with 220, 330, or 440 key lengths. These key lengths correspond to the concatenation of the key bits for 2, 3, and 4 2D maps, respectively.

The use of the 2D map allows a simple implementation in hardware because only sums and multiplications are used. We present here a software implementation in a high-performance microcontroller card STM32F746-ZG. This software implementation was easy to code because the used microcontroller allows multiplication of two 16-bit words. This software implementation will be a little hard if more 2D maps are used in parallel,

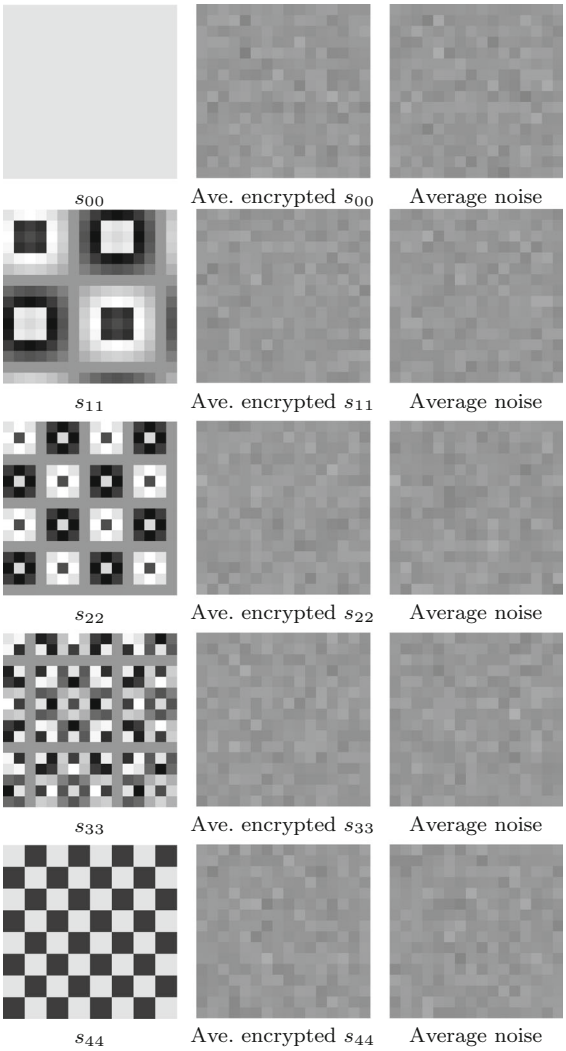


Fig. 5 In the first column are the images of the five Hartley's basic images; in the second column are the images of average 100 encrypted basic images with different key; in the third column are the images of size 16×16 resulted to average 100 random sequences

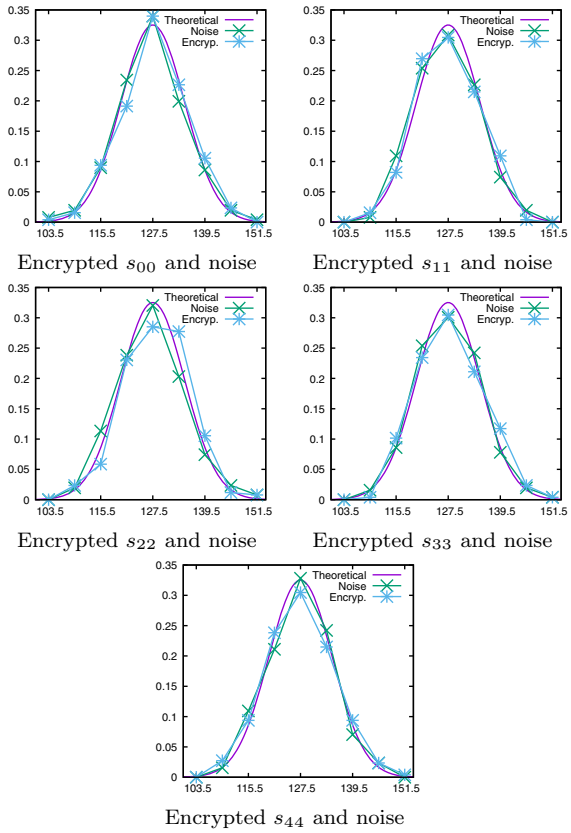


Fig. 6 Histograms of the two averaged images in Fig. 5 and the theoretical random function. It is not possible to distinguish the noise from the encrypted images

because then will need the multiplication of two numbers of 32, 48, or 64 bits, when 2, 3, or 4 maps are used in parallel, respectively.

Any information, in our case images, without care of their gray tone's probability distribution, will have a uniform distribution after performing their XOR operation [29] with the bits of the generated random sequence. Thus, the security of the encrypted images depends of the randomness of the PRNG. We prove here that 2D maps sequences are random by performing NIST and TestU01 statistical tests.

All statistical tests are applied to the images in the literature [10,17,22,26,33], as the image histogram, correlation coefficients, number of pixel change rate (NPCR), unified average changing intensity (UACI), etc., all they are in reality applied to the random numbers. Such statistical tests are not applied to measure any image statistical property. We claim such statistical tests should be removed from future reports.

We also claim here that the key is secure, because it could be impossible to discover the key from the input data.

The pseudo-dot product (PDP) is used as the basis to implement the hash function, and this is also a message authentication code; the PDP is secure because it is a universal hash function [14]. This kind of functions proved that for any message mapped into short outputs, the probability of any pair of messages collide, i.e., that both have the same hash value, is small. And this probability does not depend on any particular distribution

of the input data but only in the choice of the used universal hash function [14,28]. Of course, other universal hash functions can be used to implement a MAC [2,4,16].

7 Conclusion

We used the 2D chaotic map to build a pseudo-random number generator. This generator uses integer arithmetic with 64-bit numbers. The generated random sequences passed all the NIST 800-22-a and Test01 tests with 100 sequences of 10^6 bits. We used the PRNG to encrypt images. We proposed a keyed hash function based on the pseudo-dot product, and this keyed hash (or MAC) can be used as an authentication tag to the transmitted or stored information. The security depends on the PRNG and not of any distribution of the data in the images. Also, the keyed hash security depends on the choice of the universal hash function, and not in any particular distribution of the gray tones in the images. Potential future research directions could be: (1) To verify if the same approach to generate random numbers can be applied with more simple maps. This point is important because one possible application is in security on the Internet of Things, where used devices are very restricted in computational power. (2) Try other possible keyed hash functions. This point is theoretically possible but needs to be verified. (3) We proposed the basic keyed hash construction but not a full functional protocol implementation that uses it.

Acknowledgements Authors would like to thank the anonymous reviewers for their valuable comments which have helped to improve the quality of this article.

Declaration

Conflict of interest The authors declare that they have no conflict of interest.

References

1. Al-Hazaimeh, O.M., Al-Jamal, M.F., Alhindawi, N., Omari, A.: Image encryption algorithm based on Lorenz chaotic map with dynamic secret keys Neural Comput. Appl. 31, 2395–2405 (2019). https://doi.org/10.1007/s00521-017-3195-1
2. Bibak, K., Kapron, B., Srinivasan, V., Tóth, L.: On a variant of multilinear modular hashing with applications to authentication and secrecy codes. In: 2016 International Symposium on Information Theory and Its Applications (ISITA), pp. 320–324 (2016)
3. Chakraborti, A., Chattopadhyay, A., Hassan, M., Nandi, M.: TriviA: A fast and secure authenticated encryption scheme. Cryptology ePrint Archive, Report 2015/590 (2015). https://eprint.iacr.org/2015/590
4. Chakraborti, A., Chattopadhyay, A., Hassan, M., Nandi, M.: TriviA and uTriviA: two fast and secure authenticated encryption schemes. J. Cryptogr. Eng. **8**(1), 29–48 (2018). https://doi.org/10.1007/s13389-016-0137-2
5. Chakraborty, D., Mancillas-Lopez, C., Sarkar, P.: Stes: A stream cipher based low cost scheme for securing stored data. Cryptology ePrint Archive, Report 2013/347 (2013). https://eprint.iacr.org/2013/347
6. De Micco, L., Antonelli, M., Larrondo, H.: Stochastic degradation of the fixed-point version of 2D-chaotic maps. Chaos Solitons Fractals **104**, 477–484 (2017). https://doi.org/10.1016/j.chaos.2017.09.007
7. de la Fraga, L., Torres-Pérez, E., Tlelo-Cuautle, E., Mancillas-López, C.: Hardware implementation of pseudo-random number generators based on chaotic maps. Nonlinear Dyn. **90**(3), 1661–1670 (2017). https://doi.org/10.1007/s11071-017-3755-z
8. Ecuyer, L., Simard, R.: TestU01: A C library for empirical testing of random number generators. ACM Trans. Math. Softw. **33**(22) (2007). http://simul.iro.umontreal.ca/testu01/tu01.html
9. Gan, Z., Chai, X., Yuan, K., Lu, Y.: A novel image encryption algorithm based on LFT based S-boxes and chaos. Multimed. Tools Appl. **77**(7), 8759–8783 (2018)
10. Gao, H., Zhang, Y., Liang, S., Li, D.: A new chaotic algorithm for image encryption. Chaos Solitons Fractals **29**(2), 393–399 (2006). https://doi.org/10.1016/j.chaos.2005.08.110
11. García-Guerrero, E., Inzunza-González, E., López-Bonilla, O., Cárdenas-Valdez, J., Tlelo-Cuautle, E.: Randomness improvement of chaotic maps for image encryption in a wireless communication scheme using PIC-microcontroller via Zigbee channels. Chaos Solitons Fractals **133**, 109646 (2020). https://doi.org/10.1016/j.chaos.2020.109646
12. Gonzalez, O., Woods, R.: Digital Image Processing Fourth Edition. Pearson Education Limited, London (2018)
13. Guillén-Fernández, O., Meléndez-Cano, A., Tlelo-Cuautle, E., Núñez-Pérez, J.C., de Jesus Rangel-Magdaleno, J.: On the synchronization techniques of chaotic oscillators and their FPGA-based implementation for secure image transmission. PLoS ONE **14**(2), e0209618 (2019)
14. Halevi, S., Krawczyk, H.: MMH: Software message authentication in the gbit/second rates. In: Biham, E. (ed.) Fast Software Encryption, pp. 172–189. Springer, Berlin Heidelberg (1997). https://doi.org/10.1007/BFb0052345
15. Kadir, R., Shahril, R., Maarof, M.A.: A modified image encryption scheme based on 2D chaotic map. In: International Conference on Computer and Communication Engineering (ICCCE'10), pp. 1–5 (2010). https://doi.org/10.1109/ICCCE.2010.5556842
16. Lemire, D., Kaser, O.: Strongly universal string hashing is fast. Comput. J. **57**(11), 1624–1638 (2018). https://doi.org/10.1093/comjnl/bxt070
17. Li, B., Liao, X., Jiang, Y.: A novel image encryption scheme based on improved random number generator and its

implementation. Nonlinear Dyn. **95**(3), 1781–1805 (2019). https://doi.org/10.1007/s11071-018-4659-2
18. Li, C., Lin, D., Lü, J., Hao, F.: Cryptanalyzing an image encryption algorithm based on autoblocking and electrocardiography. IEEE Multimed. **25**(4), 46–56 (2018). https://doi.org/10.1109/MMUL.2018.2873472
19. Li, C., Tan, K., Feng, B., Lu, J.: The graph structure of the generalized discrete Arnold's cat map. IEEE Trans. Comput. **1**, 1 (2021). https://doi.org/10.1109/TC.2021.3051387
20. Li, C., Zhang, Y., Xie, E.Y.: When an attacker meets a cipher-image in 2018: a year in review. J. Inf. Secur. Appl. **48**, 102361 (2019). https://doi.org/10.1016/j.jisa.2019.102361
21. Li, J., Liu, H.: Colour image encryption based on advanced encryption standard algorithm with two-dimensional chaotic map. IET Inf. Secur. **7**(4), 265–270 (2013)
22. Liu, H., Wang, X., kadir, A.: Image encryption using dna complementary rule and chaotic maps. Appl. Soft Comput. **12**(5), 1457–1466 (2012)
23. Liu, W., Sun, K., Zhu, C.: A fast image encryption algorithm based on chaotic map. Opt. Lasers Eng. **84**, 26–36 (2016). https://doi.org/10.1016/j.optlaseng.2016.03.019
24. Liu, Z., Wang, Y., Zhao, Y., Zhang, L.: A stream cipher algorithm based on 2D coupled map lattice and partitioned cellular automata. Nonlinear Dyn. **101**, 1383–1396 (2020). https://doi.org/10.1007/s11071-020-05804-2
25. Nosrati, K., Volos, C., Azemi, A.: Cubature Kalman filter-based chaotic synchronization and image encryption. Sig. Process. Image Commun. **58**, 35–48 (2017)
26. Pareek, N., Patidar, V., Sud, K.: Image encryption using chaotic logistic map. Image Vis. Comput. **24**(9), 926–934 (2006). https://doi.org/10.1016/j.imavis.2006.02.021
27. Rukhin, A., Soto, J., Nechvatal, J., Smid, M., Barker, E., Leigh, S., Levenson, M., Vangel, M., Banks, D., Heckert, A., Dray, J., Vo, S.: A statistical test suite for the validation of random number generators and pseudo random number generators for cryptographic applications (2010). Software and SP800-22rev1a.pdf file, last date checked: Feb 28th, 2020
28. Sarkar, P.: A new multi-linear universal hash family. Cryptology ePrint Archive, Report 2008/216 (2008). https://eprint.iacr.org/2008/216
29. Sarkar, P.: Modes of operations for encryption and authentication using stream ciphers supporting an initialisation vector. Cryptogr. Commun. **6**(3), 189–231 (2014). https://doi.org/10.1007/s12095-013-0097-7
30. Sprott, J.: Automatic generation of strange attractors. Comput. Graph. **17**(3), 325–332 (1993). https://doi.org/10.1016/0097-8493(93)90082-K
31. Vaidyanathan, S., Akgul, A., Kaçar, S., Çavuşoğlu, U.: A new 4-D chaotic hyperjerk system, its synchronization, circuit design and applications in RNG, image encryption and chaos-based steganography. Eur. Phys. J. Plus **133**(2), 46 (2018)
32. Wong, K.W., Sin-Hung Kwok, B., Law, W.S.: A fast image encryption scheme based on chaotic standard map. Phys. Lett. A **372**(15), 2645–2652 (2008)
33. Yildiz, M.Z., Boyraz, O., Guleryuz, E., Akgul, A., Hussain, I.: A novel encryption method for dorsal hand vein images on a microcomputer. IEEE Access **7**, 60850–60867 (2019)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.